COMP1035-FSE: UML Diagrams Explained

Table of Contents

Software Requirements Brief *2*

Textual Analysis *4*

Table 1: Actors 4

Table 2: Use Cases 4

Use Case Diagram *6*

Descriptions 6

Key Qualities of a Good Use Case Diagram 7

Example of a Good Use Case Diagram for an HPMS 8

Persona *9*

Descriptions 9

Key Qualities of a Good Persona 9

Example of a Good Persona for an HPMS (Text-Based – Should be in Visual Format) 10

Examples of a Good Persona of Different Software Brief (Good Visual Format) 11

Example of a Template Suited for Marketing and Design Teams 12

Activity Diagram *13*

Descriptions 13

Key Qualities of a Good Activity Diagram 14

Example of a Good Activity Diagram for an HPMS 15

Sequence Diagram *16*

Descriptions 16

Key Qualities of a Good Sequence Diagram 17

Example of a Good Sequence Diagram for an HPMS 18

State Diagram *19*

Descriptions 19

Key Qualities of a Good Sequence Diagram 19

Example of a Good State Diagram for an HPMS 20

Prototyping *21*

Descriptions 21

Key Qualities of a Good Prototyping 21

Example of a Good Prototype for an HPMS (Text-Based – Should be in Visual Format) 22

Example of a Good Prototyping of Different Software Brief (Good Visual Format) 23

Software Requirements Brief

The client company requires a comprehensive hospital patient management system (HPMS) designed to streamline and automate critical hospital operations, focussing on improving patient data, appointments, medical records, and communication between healthcare professionals. The system should facilitate a seamless user experience for a variety of hospital staff, including doctors, admins, and patients, while ensuring secure access and efficient workflows.

HPMS must include key features such as patient registration, appointment scheduling, and medical record management. The system should enable patients to register their details, view their medical history, and schedule appointments. Admins will have administrative control over the system, ensuring proper management of user accounts and data, including access and update patient records, track treatment plans, and manage appointments according to their availability. They will also be responsible for generating invoices, viewing system-generated reports, and overseeing the general health and security of the system. The system will also provide hospital admins with detailed reports on hospital operations, including financial transactions and patient-related data, to facilitate better decision making.

In addition, doctors should be able to view and update patient records, while patients can access their personal data and schedule appointments. Doctors and patients can view their medical history, ensuring that both care providers and patients have access to accurate health data. The receptionist will be responsible for helping patients with their registration and appointment scheduling. The receptionist can help generate invoices for patients after checking their payment records.

The system will integrate with an external payment system that processes patient payments services. Patients will be able to make payments securely through the system, and the payment gateway will interface with HPMS to confirm and record payment transactions. Payments may also be linked to appointment scheduling, and patients are asked to pay as part of the appointment booking process.

The system must offer a clean, user-friendly interface that is intuitive and easy to navigate. A responsive design is necessary to ensure compatibility across various devices (desktop, tablet, mobile). The user interface should simplify tasks such as scheduling appointments, viewing medical records, and registering patients, with an emphasis on minimal clicks and ease of use. Each type of user will have a personalised dashboard that highlights relevant information and tasks, simplifying interactions with the system. For patients, the interface will allow easy access to appointment scheduling, medical history, and payment options. For doctors, the interface will highlight relevant patient records and treatment plans. Receptionists will have a straightforward workflow for managing patient registrations and appointments, while Admins will have access to system reports and user management features.

As a healthcare application, security is critical. HPMS should employ secure login mechanisms (e.g., username/password, multi-factor authentication) and ensure that patient data are protected through encryption and role-based access controls. Sensitive data, such as patient medical records, should be accessible only to authorised users, with strict controls in place to prevent unauthorised access.

The system will be web-based and accessible through standard web browsers, with cloud hosting for scalability and reliability. It should support integration with external services such as email or SMS for appointment reminders and payment gateways for processing patient fees, if applicable. The backend will be powered by a database (e.g., MySQL or PostgreSQL), and the frontend will use modern technologies such as React.js for an interactive user experience.

Textual Analysis

Table 1: Actors

Candidate Class	Extracted Text	Description
Doctor	"… doctors should be able to view and update patient records..."	Healthcare professionals who manage patient records, view medical history, prescribe treatment, and monitor patient progress.
Admin	"Admins will have administrative control over the system, ensuring proper management of user accounts and data"	Hospital staff is responsible for managing hospital operations, user access, patient data, and system integrity.
Patient	"Patients can access their personal data and schedule appointments."	Individuals who register their information, book appointments, and access their medical records.
Receptionist	"The receptionist will be responsible for …"	Administrative staff who handle patient registration, schedule appointments, and generate invoices.

Table 2: Use Cases

Candidate Class	Extracted Text	Description
Register Patient	"HPMS must include key features such as patient registration... Patients should be able to register their details..." "… receptionist will be responsible for helping patients with their registration and appointment scheduling …"	A process where patients provide their personal information and medical history (or to receptionists who can help) to register in the system.
Schedule Appointment	"… patients can access their personal data and schedule appointments." "… receptionist will be responsible for helping patients with their registration and appointment scheduling …"	A feature that allows patients or receptionists to book, modify, or cancel appointments and allows receptionists to manage their schedules.

Manage Medical Records	"Admins will have administrative control over the system, ensuring proper management of user accounts and data ..."	A process that allows admins to view and update patient medical history, treatment plans, and diagnoses.
Login	"HPMS should employ secure login mechanisms (e.g., username/password, multi-factor authentication) and ensure that patient data is protected through encryption..."	Implement secure login mechanisms and encryption to protect patient data and prevent unauthorised access.
View Medical History	"Doctors and patients can view medical history, ensuring that both care providers and patients have access to accurate health data"	A process for doctors and patients to view the patient's medical history, including diagnoses, treatments, and prescriptions.
Generate Invoice	"Admins … will be responsible for generating invoices …" "The receptionist can help generate invoices for patients ..."	A feature that allows receptionists and admins to generate invoices for services provided to patients.
View Reports	"Admins … will be responsible for … viewing system-generated reports, …."	A process in which admins can view various reports from the system related to patients, appointments, and financial information.
Make Payment	"Patients will be able to make payments securely through the system."	A process in which patients can make payments through the external payment system.
Check Payment Record	"The receptionist can … after checking their payment records."	A feature that allows receptionists to confirm patient payments before generating the invoices.

Use Case Diagram

Descriptions

A **use case diagram (UCD)** is a powerful tool in software design and system modelling, used to represent the functional requirements of a system from a user's perspective. A good quality UCD serves as a clear and visual representation of system interactions, helping both technical and non-technical stakeholders understand the core functionality of a system.

In a UCD, there are **FOUR** elements:

- **Actor**: Is any entity that interacts with the system. It represents a role played by a user, another system, or a component that interfaces with the system being modelled. The actor can be a person (e.g., patient, doctor, admin), an external system (e.g., payment gateway), or an internal system that communicates with the system.
- **Use case**: The use cases represent actions that actors can perform. Each use case should be written in the active form and present tense, following the format of "verb + noun".
- **Relationship**: The relationships between actor and use case(s), and between use cases.
- **Note**: A note is intended to describe a use case in detail, particularly if you want to emphasise important aspects of that use case.

There are four types of **relationships in UCD**:

- **Associations**: Lines connecting **actors to use cases**, showing interaction.
- Two types of **relationships between use cases**:
    - ***<<extend>>***: This indicates an **optional** use case that follows the base use case. This relationship is represented by **a dashed arrow that points from the extending use case to the base use case**. In other words, when a use case may include the functionality of another use case under certain conditions (e.g., "Make Payment" may extend "Schedule Appointment").
    - ***<<include>>***: This signifies a **mandatory** use case that is part of the base use case. This relationship is represented by **a dashed arrow that points from the base use case to the included use case**. In other words, when a use case always includes the functionality of another use case (e.g., "Generate Invoice" always includes "View Patient Information").
- **Generalisation:** When one **actor inherits the behaviour of another actor**.

A high-quality UCD should be **clear**, **simple**, and **well-structured**, following UML conventions and accurately representing the system's functionality. It should include the right level of detail, with clearly defined actors, use cases, and relationships, all while adhering to best practices in terms of visual clarity and naming conventions. **Professional-level diagrams excel in usability, readability, and conveying the core functions of the system efficiently.**

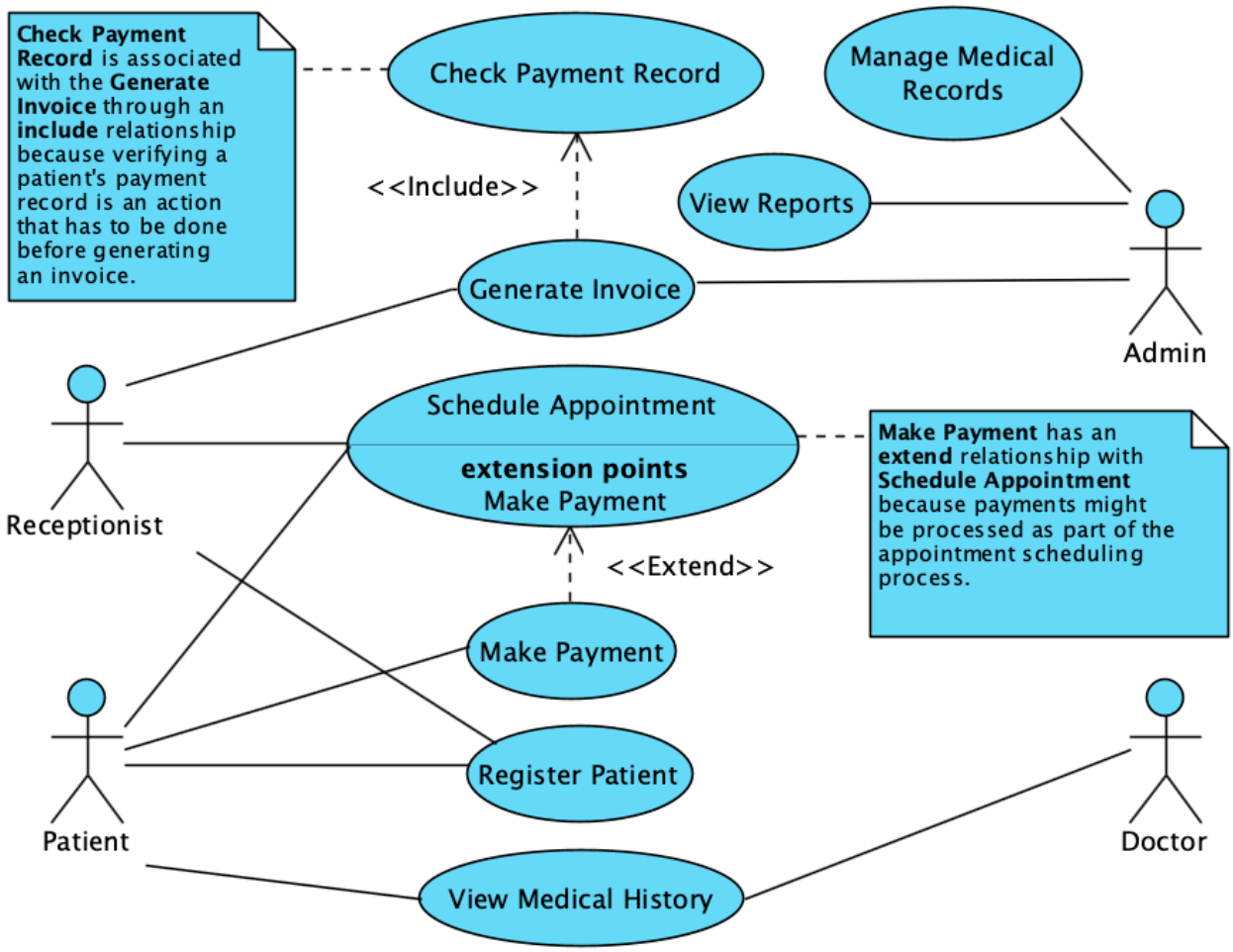
Key Qualities of a Good Use Case Diagram

- **Clarity and Simplicity**
  - A good UCD should clearly represent the system's use cases without excessive complexity. It should avoid overloading the diagram with too many use cases, actors, or relationships, making it easy for stakeholders to understand.
  - **Example**: If the system involves patient registration, appointment scheduling, and medical history tracking, the diagram should reflect those primary use cases without unnecessary details or additional layers.
- **Correct Use of Symbols**
  - The diagram should adhere to the standard UML symbols for use cases, actors, and relationships. Actors should be represented as stick figures, use cases as ellipses, and relationships (associations, include, extend, generalisation) should be represented with appropriate lines and arrows.
  - **Example**: A Receptionist (actor) interacting with a "Register Patient" use case would be depicted with a Receptionist (stick figure) connected to the "Register Patient" use case (ellipse).
- **Well-Defined Actors**
  - **Actors** in a UCD represent users or other systems that interact with the system being modelled. Good UCD will clearly define actors by their roles, ensuring that each actor is specific and distinct.
  - **Example**: Instead of a generic actor like "User", you might use specific actors such as "Patient", "Doctor", "Receptionist", and "Admin" to clarify who is performing what actions in the system.
- **Appropriate Use Cases**
  - The UCD represent different and meaningful functionality. Good UCD include only those use cases that are necessary for the core operations of the system.
  - **Example**: For HPMS, useful use cases might include "Register Patient", "Schedule Appointment", "View Medical History", and "Generate Invoice".
- **Logical Relationships**
  - The **relationships** between actors and use cases, as well as among use cases, should be logical and clearly defined.
  - **Example**: The "Patient" actor may be associated with the "View Medical History" use case, while "Admin" may be associated with the "Generate Invoice".
- **Clear System Boundary**
  - The **system boundary** should be clearly defined, showing which use cases belong to the system and which are outside it. The system boundary box helps stakeholders distinguish what is included in the system and what is external.
  - **Example**: The HPMS boundary would clearly define the system functionalities (like patient registration, scheduling, etc.) and show that "Payment Gateway" is outside the system, represented by an external system.
- **Appropriate Level of Abstraction**

- A good UCD should be abstract enough to avoid unnecessary detail yet detailed enough to provide useful information about system functionality. It should capture high-level user goals without delving into implementation specifics (**no workflow, steps, or phases between use cases**).
- **Example**: Instead of showing every step in a registration process, the diagram might simply represent the use case "Register Patient" as a single, high-level use case, **leaving the detailed steps for other modelling methods or diagrams such as activity diagrams**.

- **Consistency and Naming Conventions**
  - Consistent and descriptive naming conventions for actors and use cases help avoid confusion. Use case names should describe the action being performed, **usually in a verb-noun format** (e.g., "View Medical History").
  - **Example**: An actor should be named "Patient" instead of "User" to clarify their role in the system, and a use case should be named "View Medical History" instead of "Medical History Viewing and Checking" (incorrect – two actions and not in verb-noun format).

Example of a Good Use Case Diagram for an HPMS



Persona

Descriptions

A **good persona** is one that reflects real users with clear goals, behaviours, and pain points, grounded in research. It should help guide design decisions and align product development with real user needs. **A professional-level persona excels in depth, clarity, completeness, and actionability while also being realistic and visually engaging.**

Key Qualities of a Good Persona

- **Realistic and Believable**
  - A persona should reflect a real-world user with characteristics, goals, and behaviours that feel authentic. The persona should be based on data (user research, interviews, surveys) rather than assumptions or stereotypes.
  - **Example**: A persona for a HPMS might include details like the age of the user, their level of tech literacy, their role in the healthcare process (e.g., doctor, patient) and their needs and frustrations with the current system.
- **Clear Goals and Motivations**
  - A persona should define the user's **goals**, **needs**, and **motivations** that drive their actions in the context of the product. These goals will help you understand why the user engages with the system and what they expect to achieve.
  - **Example**: A persona could have the goal of "reducing time spent on patient paperwork" or "ensuring timely and error-free appointment scheduling".
- **Contextual and Relevant Information**
  - The persona should include information that is directly relevant to the product being designed. This could include the user's environment, pain points, tasks, or other contextual factors that influence how they interact with the system.
  - **Example**: If the system is meant for hospital staff, the persona could include working conditions such as high-pressure environments or time constraints that affect how they use the product.
- **Behavioural Characteristics**
  - A persona should capture the typical behaviours and actions of the user group it represents. This includes how they typically interact with technology, what devices they use, and how they make decisions.
  - **Example**: The persona might describe whether the user prefers mobile apps, works long hours, or is highly detail-orientated when entering data.
- **Demographics and Psychographics**
  - **Demographics**: Basic personal information such as age, gender, education, and role that helps provide context for the persona's decisions and behaviour.

    - **Psychographics**: Information about the persona's values, interests, attitudes, and lifestyle, which can help explain their decisions and preferences beyond basic demographics.
    - **Example**: Age, tech-savvy level, education, whether they are risk averse or how they value convenience over time.
- **Challenges and Pain Points**
    - A good persona includes information about the **pain points** or **challenges** that the user faces, particularly related to the area the product is aiming to improve. Understanding these pain points helps design solutions that address real user problems.
    - **Example**: A persona might have the challenge of "struggling with an outdated system that requires excessive manual entry" or "frustration with miscommunication in scheduling appointments".
- **Persona Environment**
    - A good persona includes information about the **environment** in which they will use the system. This could include physical location (e.g., office, hospital ward), technology (e.g., using a desktop, tablet, or mobile), or even social setting (e.g., collaborative environment, working alone).
    - **Example**: For a healthcare persona, the environment could include working long shifts, limited access to technology in certain areas, or collaboration with a team in a hectic hospital setting.
- **Names and Personal Details**
    - To make personas more relatable and memorable, it is helpful to give them names, photos, and other personal details. This makes them feel like real people and can make the design team more empathetic to the persona's needs.
    - **Example**: "Dr. Emily Yang, an experienced paediatrician who works long hours in a busy city hospital".

Example of a Good Persona for an HPMS (Text-Based – Should be in Visual Format)

Name: Dr. Emily Yang
Role: Paediatrician at City Hospital
Age: 42
Education: MD, Board-Certified Paediatrician
Technological Proficiency: Intermediate (comfortable using the current hospital's electronic medical record system)
Goals:

- To reduce administrative burden (e.g., paperwork, patient data entry) to focus more on patient care.
- To schedule patient appointments without conflicts and with minimal errors.
- To access patient medical history quickly and efficiently.

Challenges:

- Frustrated with the current patient registration system, which is slow and prone to errors.
- Often has limited time to update patient records due to the high volume of patients.
- Need to juggle multiple devices and systems during rounds.

Motivations:

- Wants a more efficient workflow to ensure more time with patients.
- Values reliability and accuracy in patient data.
- Concerned about data security and patient privacy.

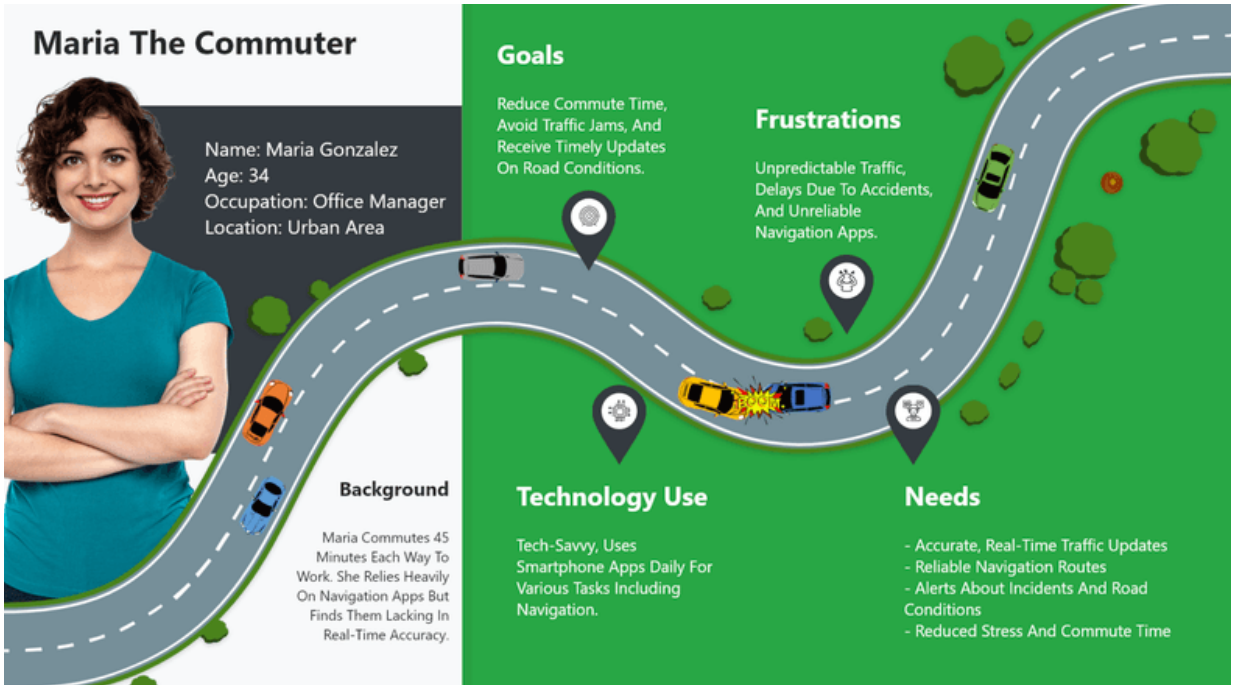
Environment:

- Work in a fast-paced hospital environment, often working long shifts.
- Uses desktop computers and tablets to manage patient data.
- Collaborates with admins staff, and other doctors during rounds.

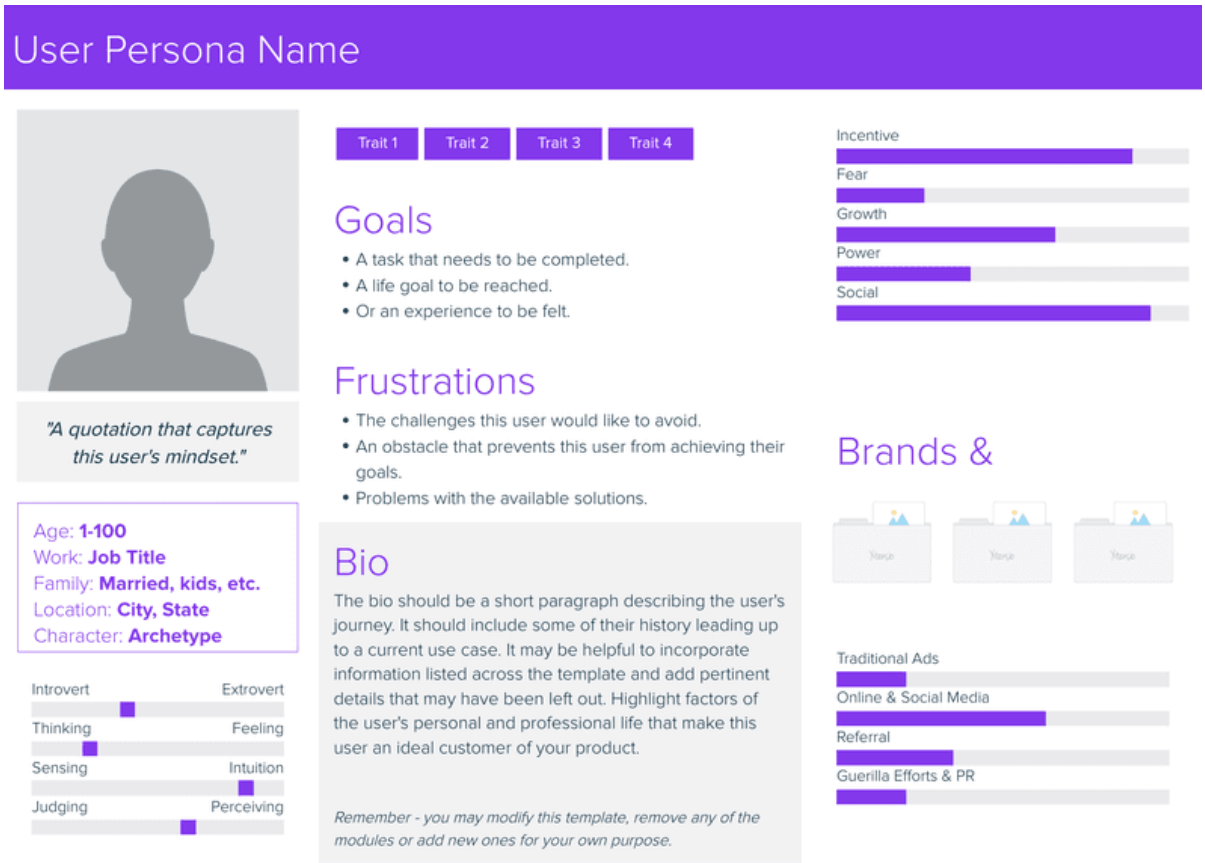
Behavioural Traits:

- Prefers to receive updates in real time and on the go, using mobile devices or tablets when possible.
- Makes quick decisions based on available data but needs reliable and clear information.

Examples of a Good Persona of Different Software Brief (Good Visual Format)



Example of a Template Suited for Marketing and Design Teams



Activity Diagram

Descriptions

An activity diagram is a type of behavioural diagram in UML that models the flow of activities in a system, focussing on the **flow of control and the sequence of actions or steps that occur within a process**. Activity diagrams are particularly useful for describing workflows, business processes, and complex operations in software design. A good quality activity diagram is clear, concise, and accurately models the system activities and their interactions.

Typically, **an activity diagram should be associated with ONE use case**, although there are instances where an activity diagram may involve several use cases. In software development, there should ideally be only a use case diagram but multiple activity diagrams. An activity diagram consists of **SEVEN** basic elements:

- **Initial/Final Node**: The initial node marks the starting point of the activity and is represented by a black circle. A final (or end) node signifies the conclusion of the activity and is denoted by an outlined black circle. **Each activity diagram should have only ONE initial node and ONE final node**.
- **Activity Node**: An activity is a task that never produces an outcome by itself and is represented by round-edged rectangles. The text within the activity node is bold and aligned to the left by default.
- **Action Node**: An action denotes a step in the activity that generates a result and is illustrated by round-edged rectangles. The text within the action node is centre aligned by default.
- **Decision Node**: This node signifies a conditional branch in the flow and is represented by a diamond shape. It includes a **SINGLE input and TWO or MORE outputs**.
- **Control Flow**: The control flow connectors indicate the sequence of steps (activity or action) in the diagram.
- **Swimlane**: Swimlanes indicate which activities are carried out by specific organisations or actors in the activity diagram. These lanes act as boundaries within which the activities of a particular organisation are depicted. Swimlanes are optional and are not necessary if only one organisation is involved.
- **Note**: A note is intended to describe activities in detail, particularly if you want to emphasise important aspects of those activities.

A high-quality activity diagram should provide a **clear**, **visual representation of the flow of activities in a system**, with the **correct use of UML symbols and conventions**. It should effectively represent the sequence of actions, decisions, and parallel processes and ensure that stakeholders can easily understand the workflow. **Professional-level activity diagrams excel in clarity, simplicity, and accuracy, while also maintaining consistency and using swimlanes, branching, and parallel (fork and merge) activities effectively to show responsibility across different actors or systems.**

Key Qualities of a Good Activity Diagram

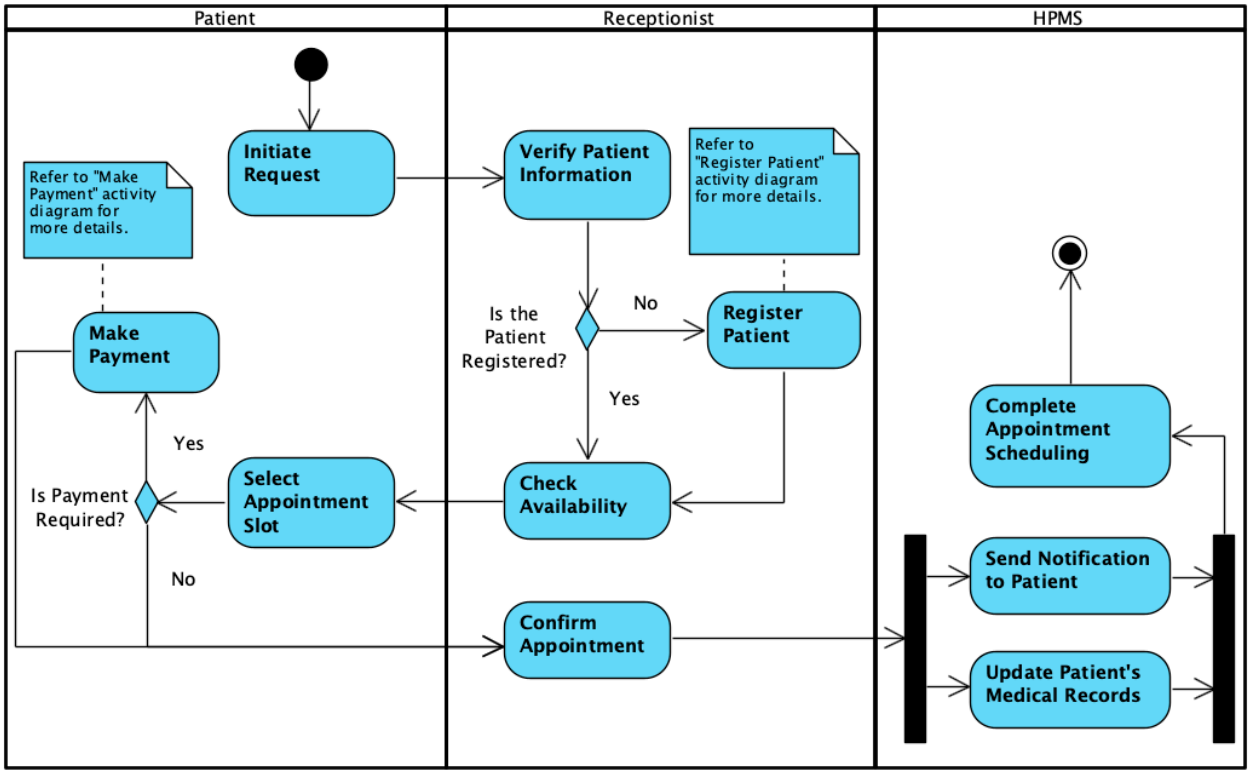
- **Clear Flow of Control**
  - A good activity diagram should clearly depict the flow of control between actions, decisions, and parallel processes. Arrows (control flows) should be used to represent the sequence of actions, to ensure that the flow of the diagram is logical and easy to follow.
  - **Example**: The activity diagram of an HPMS could show the flow from patient registration, to scheduling an appointment, to receiving treatment.
- **Correct Use of Symbols**
  - The diagram should correctly use standard UML symbols for activities, decisions, start and end points, synchronisation bars (for parallel actions), and swimlanes (if needed). Each symbol should be used appropriately, to ensure that the diagram is easy to read and interpret.
  - **Example**: Activities are represented by rounded rectangles, decisions are represented by diamonds, and initial and final nodes with solid and hollow circles, respectively.
- **Accurate Representation of Activities**
  - The diagram should accurately represent the **activities** or actions in the process being modelled. Each activity should reflect a step in the process that is clear and meaningful, contributing to the overall workflow.
  - **Example**: In the "Schedule Appointment" use case, activities could include "Verify Patient Information", "Select Available Appointment Slot", and "Confirm Appointment".
- **Clear Branching and Decision Points**
  - **Decision nodes** should clearly represent points where a process can branch based on conditions (e.g., if-else decisions). Each branch should have a clear condition labelled near the outgoing arrows to make the decision logic explicit.
  - **Example**: A decision point might ask "Is the patient registered?" and have two branches: one for "Yes" and one for "No", leading to different actions (e.g., "Register Patient" or "Check Availability").
- **Proper Use of Synchronisation (Forks and Joins)**
  - **Forks** (splitting a single flow into parallel flows) and **joins** (merging parallel flows back into one) should be used appropriately to model concurrent processes. The diagram should clearly show parallel activities where multiple tasks occur simultaneously.
  - **Example**: In the "Schedule Appointment" use case, the "Send Notification to Patient" and "Update Patient's Medical Records" activities might occur in parallel after an appointment is confirmed.
- **Swimlanes (if applicable)**

- **Swimlanes** are useful for grouping activities by the actors or systems responsible for them. If a system involves multiple roles or systems (e.g., user and system), swimlanes should be used to clarify who is responsible for each activity.
- **Example**: For "Schedule Appointment" use case, you might have swimlanes for "Patient", "Receptionist", and "HPMS", each showing their respective actions.

- **Start and End Points**
  - The **initial node** (start point) and **final node** (end point) should be clearly defined. The diagram should start with an initial node (solid circle) and end with a final node (hollow circle), ensuring that the process flow is well defined and complete.
  - **Example**: For "Schedule Appointment" use case, the process could start with the action "Patient Initiates Request" and end with "Complete Appointment Scheduling".
- **Consistency and Clarity**
  - The diagram should be consistent in terms of naming conventions, flow direction (typically top-down or left-right), and the use of symbols. The diagram should be clean, with no overlapping arrows or unclear labels that could confuse the reader.
  - **Example**: Each decision point should have a clear label indicating the decision condition, and no arrows should cross unnecessarily.

Example of a Good Activity Diagram for an HPMS

The activity diagram of the "Schedule Appointment" use case (through receptionist).



Sequence Diagram

Descriptions

A sequence diagram is another type of behavioural diagram in UML that illustrates **how objects or components in a system interact with each other over time**. Emphasis is placed on the sequence of messages exchanged between objects in a particular process or scenario. A good-quality sequence diagram helps clarify the flow of interactions, responsibilities, and timing of method calls in a system. It should provide clear, concise, and accurate information on the dynamic behaviour of a system.

Typically, **a sequence diagram is associated with a SINGLE use case**, although it may include multiple use cases. **A sequence diagram can involve ONE or MORE actors**. There are **FIVE** basic elements in a sequence diagram, including:

- **Actor**: An actor in a sequence diagram should correspond to the actor in the use case diagram. It represents the roles performed by human users, external hardware, or other entities involved in the system.
- **Lifeline**: A lifeline represents an individual component in the interaction or data of the system. It is typically represented as a class object in object-oriented programming (OOP) and is depicted by a blue rectangle placed at the top of the sequence diagram.
- **Activation**: Activation is depicted by a thin vertical rectangle and represents the period during which an element performs an operation.
- **Message**: There are four different types of messages, including:
    - **Call** Message: Represents an invocation of an operation on the target lifeline, depicted as an arrow pointing to the right.
    - **Return** Message: Indicates the passing of information back to the caller of a previous message, depicted as an arrow pointing to the left.
    - **Self**-Message: Represents the invocation of a message on the same lifeline.
    - **Recursive** Message: Indicates the invocation of a message on the same lifeline.

  You can think of a message as a method call, usually in the format of *message_name (input arguments)*.
- **Alternatives**: The alternative combination fragment is used when a choice needs to be made between two or more message sequences. It models the "**if-then-else**" logic. The type of alternatives are as follows:

Operator	Type	Description
alt	Alternative Combined Fragments	Only the one whose condition is true will execute.
opt	Optional	The fragment executes only if the supplied condition is true. Equivalent to an ***alt*** only with one trace.
par	Parallel	Each fragment is run in parallel.

loop	Loop	The fragment may execute multiple times, and the guard indicates the basis for the iteration.
region	Critical Region	The fragment can have only one thread executing it at once.
neg	Negative	The fragment shows an invalid interaction.
ref	Reference	Refers to an interaction defined on another diagram. The frame is drawn to cover the lifelines involved in the interaction. You can define parameters and a return value.
sd	Sequence Diagram	Used to surround an entire sequence diagram.

A high-quality sequence diagram should provide a **clear and logical representation of the interactions between objects in a system**. It should follow UML conventions and accurately model the sequence of messages, the timing of interactions, and the involvement of various system components. **A professional-level sequence diagram excels in clarity, correctness, and simplicity, with a focus on making interactions easy to understand for all stakeholders.**

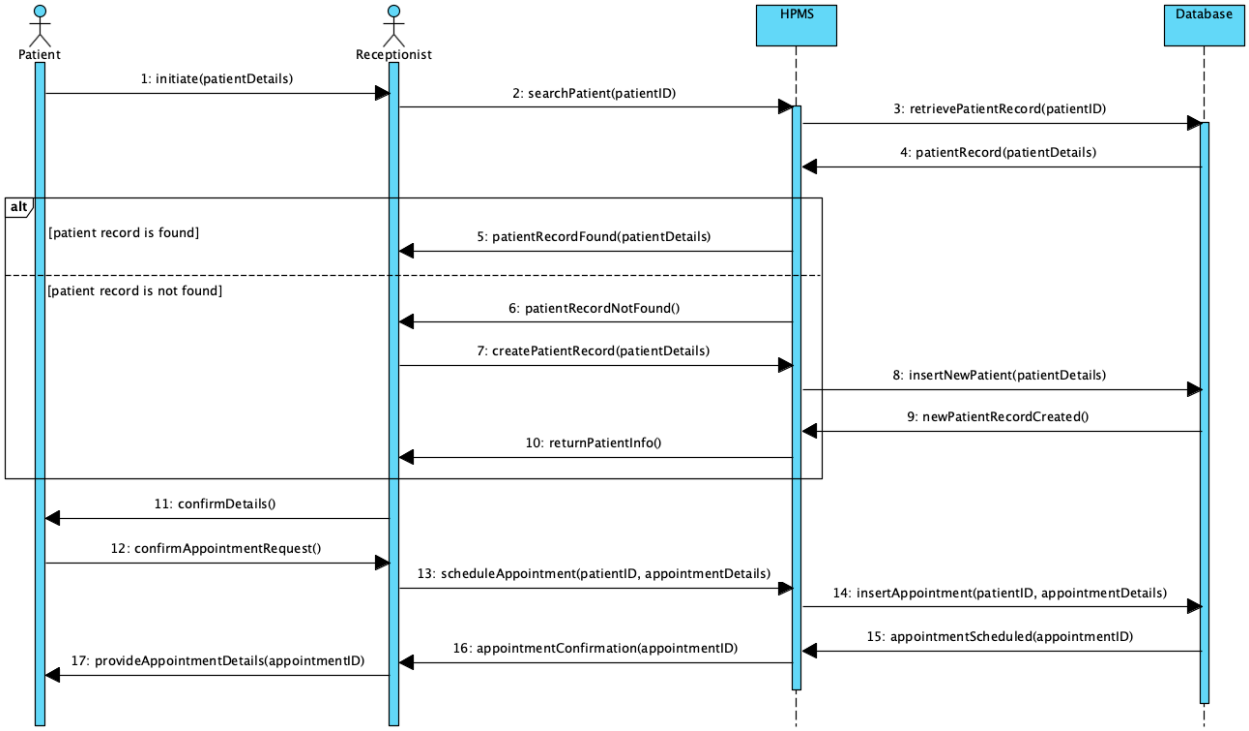
Key Qualities of a Good Sequence Diagram

- **Clear Identification of Participants (Objects/Actors)**
  - The diagram should clearly represent all the key objects (like patient, receptionist, doctor, HPMS) involved in the interaction.
  - **Example**: For HPMS, the actors might be **Patient, Receptionist, and Doctor** (following the UCD).
- **Logical Flow of Interaction**
  - The diagram should accurately represent the **sequence of actions** and interactions between objects following the natural order in which events occur.
  - **Example**: A typical flow might start when the **Patient** logs into the **HPMS** and ends when the patient receives the treatment details from the doctor.
- **Correct Representation of Message Types**
  - **Synchronous messages** (solid arrows) indicate when a system component waits for a response before continuing.
  - **Asynchronous messages** (dashed arrows) show actions where the sender does not wait for a reply.
  - **Return messages** (dashed arrows) indicate the return of data or response.
- **Use of Activation Bars**
  - **Activation bars** indicate when a particular object is **active** and engaged in processing a message or task. These bars should be used consistently to represent periods during which an object performs actions.
- **Handling of Alternative Paths (Conditional Flows)**
  - Different outcomes may arise, depending on the conditions. The diagram should include fragments like ***alt*** or ***opt*** to represent conditions or choices.
  - **Example**:

        - If the patient has an appointment, the receptionist checks the patient appointment records.
        - If the patient does not have an appointment, they may be asked to schedule one.
- **Return Messages**
    - The diagram should include **return messages** to reflect responses from the system or other objects. These should be clear and labelled with the return data (e.g., appointment confirmation, error messages).
- **Use of Loops for Repetitive Actions**
    - **Loop fragments** represent repeated actions.
    - **Example**: If the patient has multiple prescriptions, the process of retrieving and updating prescription details could be repeated.
- **Simplicity and Focus**
    - The diagram should focus on the main flow of events and avoid unnecessary complexity. Non-critical interactions should not be included if they do not contribute to the core scenario.

Example of a Good Sequence Diagram for an HPMS

The sequence diagram of the "Schedule Appointment" use case (through receptionist).



State Diagram

Descriptions

A state diagram (also called a state machine diagram) is a type of UML diagram used to **represent the states of an object and the transitions between those states**. It is particularly useful in systems where objects go through different stages or statuses in response to events or conditions. **A professional-level state diagram should clearly represent the states of an entity throughout its lifecycle, focussing on major events and transitions.** A state diagram consists of **FIVE** fundamental elements which include:

- **State**: This is represented by a rectangle with rounded corners and signifies the current condition of an object.
- **Event**: An event is an occurrence that triggers a transition, labelled above the corresponding transition arrow.
- **Guard Condition**: A guard condition is a Boolean condition that either permits or prevents a transition from occurring.
- **Transition**: This is represented by an arrow connecting one state to another, denoting a change in state.
- **Action**: An action is an executable process that leads to a change in the model state or the return of a value.

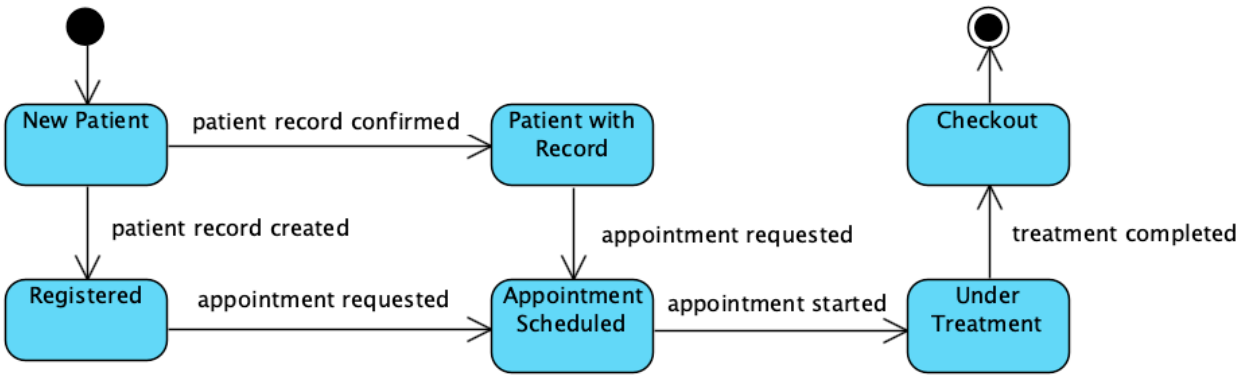
Key Qualities of a Good Sequence Diagram

- **Clear Representation of States**
  - Each **state** should clearly define a particular condition or status of the object being modelled.
  - **Example**: the **Patient** state machine might include states such as **New Patient, Registered, Appointment Scheduled, Under Treatment, and Checkout.**
- **Well-Defined Transitions**
  - The diagram should show how the system moves from one state to another, based on certain **events** or **conditions**.
  - **Example**:
    - A **Patient** moves from "Registered" to "Appointment Scheduled" when a receptionist schedules an appointment.
    - A **Patient** may transition from "Under Treatment" to "Checkout" after receiving treatment and being cleared for release.
- **Clear Event Triggers**
  - Each transition should be triggered by an **event** (an action, an input, or a change in conditions).
  - **Example**:
    - A **Patient** transitions from "New Patient" to "Registered" upon receiving basic information and creating a record in the system.

        - A **Patient** transitions from "Appointment Scheduled" to "Under Treatment" once they see the doctor.
- **Initial and Final States**
    - The diagram should have a clearly marked **initial state** (often represented with a filled circle) to indicate the starting point of the object and a **final state** (represented by a filled circle with a border) to signify the completion of the lifecycle of the object.
    - **Example**: A patient might start as "New Patient" (initial state) and move through various stages until they reach the "Checkout" state (final state).
- **Guard Conditions**
    - **Guard conditions** are Boolean expressions that must be true for a transition to occur. These conditions should be clearly indicated next to the transition arrow to show when a transition occurs.
    - **Example**: A transition from "Appointment Scheduled" to "Under Treatment" could have a guard condition like [appointment started].
- **Composite States (if applicable)**
    - If the system involves nested or hierarchical states, the diagram should include **composite states**.
    - **Example**, a "Checkout" state could be composite, containing substates like "Payment Settled" and "Report Released".
- **Consistency and Simplicity**
    - The state diagram should focus on the most important states and transitions. It should avoid unnecessary complexity, only showing the critical lifecycle states and key transitions relevant to the system operations.
- **Action and Activities**
    - **Actions** or **activities** triggered during transitions should be clearly labelled.
    - **Example**: During the transition from "Under Treatment" to "Checkout", the action might be "Treatment Completed".

Example of a Good State Diagram for an HPMS

The state diagram of the lifecycle of a patient in HPMS.



Prototyping

Descriptions

Prototyping is an iterative and interactive design process that involves **creating a working model of a system** or product to **explore ideas, test concepts, gather feedback, and refine the final product**. It helps in **visualising and validating requirements, user interactions, and functionality early in the design phase**. **A good quality prototype serves as an effective tool for communicating design ideas, testing features, and assessing usability.** The process should be flexible, allowing continuous improvements based on user input and system feedback.

Key Qualities of a Good Prototyping

- **Realistic Representation of Core Functionality**
  - The prototype should focus on the most important and critical functionalities of the system. For HPMS, this might include features such as **patient registration**, **appointment scheduling**, **patient history viewing**, and **medical record management**.
  - **Example**: A prototype for HPMS might include an interactive form for patient registration that simulates the real input process, including fields for personal information, contact details, and medical history.
- **User-Centred Design**
  - The prototype should reflect user needs and be designed with the primary users in mind. For HPMS, users could include hospital staff such as **Doctors**, **Admins**, and **Patients**. The interface should be based on the role of each user, providing an intuitive experience.
  - **Example**: For the doctor interface, the prototype might show features such as the ability to view patient records, update treatment information, and add prescriptions in a simple and clear layout.
- **Interactive Elements and Functionality**
  - A good prototype is **interactive** where it allows users to interact with various elements, such as buttons, forms, or menus, to simulate the expected user experience. For HPMS, this means allowing interactions such as registering a new patient, scheduling an appointment, and viewing or editing patient records.
  - **Example**: A clickable prototype might simulate booking an appointment by selecting available time slots and filling in the necessary details.
- **Low-to-High-Fidelity Levels**
  - **Low-fidelity prototypes** (e.g., wireframes or paper mockups) are useful in the early stages of design for quick validation of ideas and concepts.
  - **High-fidelity prototypes** (interactive models or simulations) are created later to show near-final functionality, user interfaces, and workflows.

    - **Example**: Initially, a low-fidelity prototype might include paper sketches or wireframes of the patient dashboard in the HPMS. Later, a high-fidelity prototype might be developed using a tool like Figma or Adobe XD, where the user can interact with the interface, input data, and navigate between screens.
- **Iterative Development and Feedback Incorporation**
    - Prototypes should be **iterative**, which means that they evolve based on feedback from users and stakeholders. Early feedback should be collected to validate assumptions and improve the design.
    - **Example**: After initial feedback on the HPMS prototype, changes could be made to improve usability, such as simplifying patient record navigation or clarifying medical terminology in the interface.
- **Clear User Flow and Navigation**
    - A good prototype should have a **clear user flow**, showing how users move through different screens or stages in the system. The flow should represent the typical tasks users would perform in the system, such as registering a patient, checking a medical history, or scheduling a follow-up appointment.
    - **Example**: For the HPMS prototype, the user flow could start from the **login screen**, go to the **dashboard**, proceed to **patient search**, then **view patient history**, and end with **updating medical records**.
- **Realistic Visual Design**
    - Even though prototypes may not be fully developed, they should look and feel like the actual system in terms of layout, colour scheme, fonts, and button placements. The visual design helps stakeholders imagine what the final product will look like.
    - **Example**: For the HPMS, a prototype could include realistic icons for patient profiles, appointment calendars, and medical history charts, giving users a sense of the aesthetic and usability of the final product.
- **Error Handling and Edge Case Representation**
    - A good prototype should simulate how the system will handle errors or unexpected situations, such as incorrect data input, missing information, or system downtime.
    - **Example**: If a user tries to book an appointment for an unavailable time slot in the HPMS prototype, a clear error message should appear, indicating the issue (e.g., "Time slot not available, please select another.").

Example of a Good Prototype for an HPMS (Text-Based – Should be in Visual Format)

Prototyping Tool:

- **Tool**: Figma or Adobe XD
- **Prototype Type**: High-fidelity, interactive prototype

Key Features:

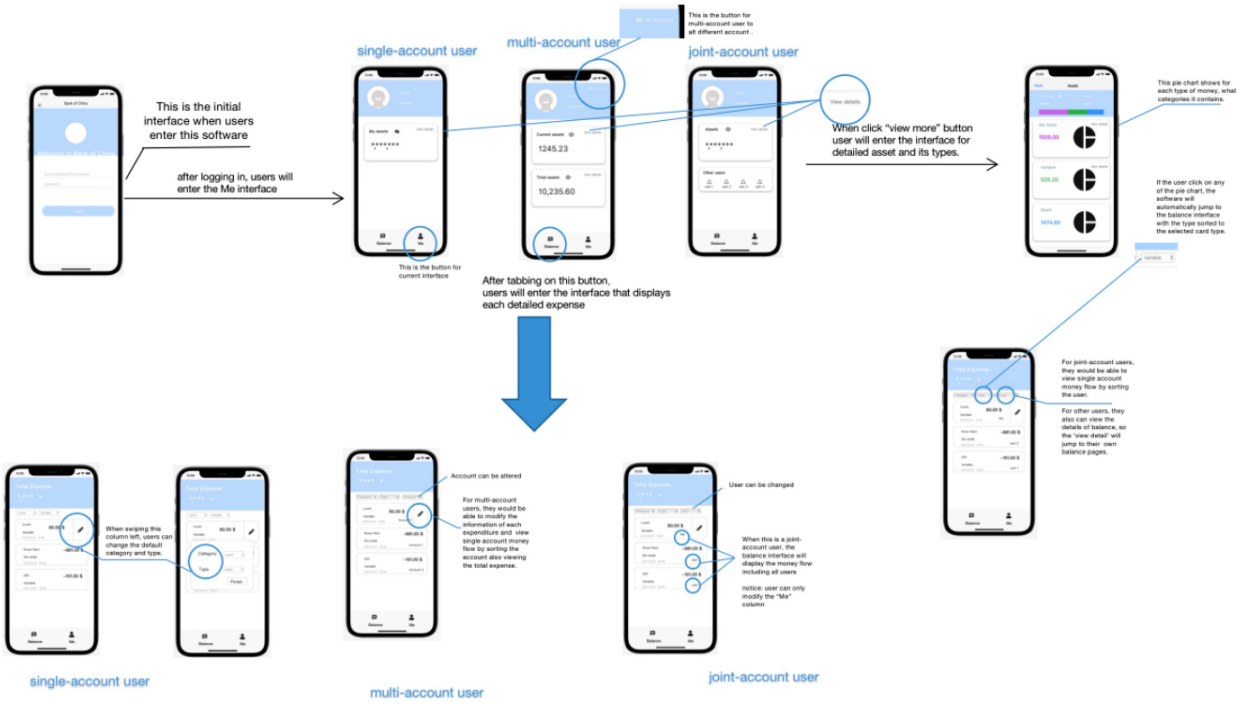
1. **Login Screen**: Users (doctors, admins) can log in with role-specific credentials.

2. **Patient Dashboard**: Shows a summary of the patient personal details, medical history, current treatment, and appointments.
3. **Appointment Scheduling**: Allows users to search for available time slots and schedule appointments.
4. **Patient Registration**: A form for adding a new patient, collecting basic information and medical history.
5. **Medical Records Viewing**: Displays patient medical records, including previous treatments, diagnoses, and prescriptions.
6. **Error Handling**: If invalid data are entered in the patient registration form, a clear error message appears: "Please enter a valid email address."

User Flow:

1. **Login Screen**: The user logs in based on their role (doctor, admin).
2. **Patient Dashboard**: After logging in, users see the dashboard with quick access to patient data.
3. **Register New Patient**: Admin can add a new patient through an interactive form.
4. **Schedule Appointment**: Patients can choose the available timeslots and confirm an appointment.

Feedback and Refinements:

- **Initial User Testing**: Early feedback indicated that the appointment scheduling interface was too disorder. Based on this, the design was simplified by grouping similar options (date, time, doctor) into dropdown menus.

Example of a Good Prototyping of Different Software Brief (Good Visual Format)